Assignment: The "Isekai" Guild Master – A Multi-Agent Quest Routing System

Objective: Design a multi-agent workflow that processes chaotic "quest requests" from fantasy villagers, cross-references them against a mock database of monster weaknesses, and outputs a safe, strategically sound party of adventurers to complete the mission.

The Problem Statement

In a typical fantasy anime, the Adventurer's Guild receives panicked, unstructured requests from citizens (e.g., *"Giant slimes are eating the harvest!"* or *"A shadow dragon is in the mountains!"*). You must build an automated Multi-Agent System (MAS) that parses these requests, researches the monsters, and drafts a battle plan. The system must include a safety loop to ensure low-level adventurers aren't sent to their doom.

Agent Roles & Architecture

Students must implement a state-graph architecture (e.g., using LangGraph, CrewAI, or raw Python loops) containing the following four agents:

1. **The Guild Receptionist (Structured Output Agent):**
    - **Input:** Raw string text of the villager's plea.
    - **Task:** Extract the core entities using prompt engineering to force a JSON output.
    - **Output Schema:** `{"enemy_type": str, "environment": str, "urgency_level": "Low" | "Medium" | "High"}`
2. **The Grand Library Agent (Tool-Calling Agent):**
    - **Input:** The extracted `enemy_type`.
    - **Task:** Must use a provided mock Python function (tool) called `search_monster_manual(enemy)`. This tool queries a hardcoded Python dictionary that returns the monster's weaknesses (e.g., "Slime -> Weak to Fire, Immune to Physical").
    - **Output:** A synthesis of the quest details and the monster's weaknesses.
3. **The Guild Strategist (Logic & Planning Agent):**
    - **Input:** The synthesized monster data.
    - **Task:** Select a 3-person party from a hardcoded list of available anime-trope adventurers (e.g., *Megumin - Explosion Mage*, *Darkness - Heavy Tank*, *Kirito - Dual Swordsman*, *Aqua - Healer*). Draft a 3-step battle plan exploiting the monster's weakness.
4. **The Guild Master (Cyclic QA / Supervisor Agent):**
    - **Input:** The drafted battle plan and party composition.

    - **Task:** Review the plan against strict safety rules. (e.g., *Rule 1: If fighting an undead/shadow monster, a Healer must be present. Rule 2: If the urgency is "High", the party cannot consist only of beginner classes.*)
    - **Logic:** If the plan fails the rules, the Guild Master outputs a critique and loops the state back to the **Guild Strategist** for a re-plan. If it passes, the plan is approved.

Technical Constraints & Requirements

- **State Management:** The architecture must pass a shared `State` object (dictionary or Pydantic model) between agents, tracking the original query, the JSON extraction, the tool response, and the iteration count.
- **The Feedback Loop:** The system must demonstrate a cyclic edge. The Guild Master must be able to reject a plan and force the Strategist to try again at least once before allowing the code to terminate.

Test Cases

Your MAS must correctly process the following inputs:

1. *"Help! A swarm of gelatinous physical-resistant slimes is destroying our wheat fields! They are everywhere!"*
    - **Expected:** Strategist must select a magic-user (like a Fire Mage); Guild Master should approve on the first pass.
2. *"The legendary Shadow Lich has awakened in the crypts! The capital will fall by midnight!"*
    - **Expected:** High urgency. If the Strategist forgets to include a Healer/Holy magic user for the undead, the Guild Master **must** catch it, trigger a loop, and force a party revision.
3. *"A giant goblin is blocking the bridge. It's just sitting there eating apples."*
    - **Expected:** Low urgency, simple physical combat party is acceptable.